#4

Group Anagrams

Arrays & Hashing | LeetCode 49 | NeetCode "Group Anagrams"

PROBLEM

Given an array of strings `strs`, group the anagrams together. You can return the answer in any order. An anagram is a string that contains the exact same characters as another string, but the order of the characters can be different.

```
strs = ["eat","tea","tan","ate","nat","bat"] → [["bat"],["nat","tan"],["ate","eat","tea"]]
```

```
strs = ["x"] → [["x"]]
```

CORE IDEA

Goal

Partition a list of strings into groups where every string in a group is an anagram of every other string in that group.

Key insight

Anagrams always produce the same letter-frequency signature, so instead of comparing every pair of strings ($O(n^2)$ work), compute one canonical "fingerprint" per string and use it as a hash map key — strings with the same fingerprint land in the same bucket automatically.

> Trigger: “Group things that are the same letters rearranged” → build a frequency-count signature and bucket by it (Hash Map).

OPTIMAL SOLUTION

Data structure — Hash Map (Python `defaultdict(list)`) keyed by a 26-length character-count tuple

1. Create `results` as a `defaultdict(list)` so a brand-new key automatically starts with an empty list.
2. For each string `s` in `strs`, build a 26-length `count` array (one slot per lowercase letter) and increment `count[ord(c) - ord('a')]` for every character `c` in `s`.
3. Convert `count` to a `tuple` (lists aren't hashable, so they can't be dict keys) and use it as `key`.
4. Append `s` to `results[key]` — every anagram of `s` produces the exact same `count` tuple, so they all land in the same bucket.
5. Return `list(results.values())` — the grouped anagrams, in any order.

O(n · k) — n strings, each of average length k	O(n · k)
TIME	SPACE

CODE

```python
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        results = defaultdict(list)

        for s in strs:
            count = [0] * 26

            for c in s:
                count[ord(c)-ord('a')] += 1

            key = tuple(count)
            results[key].append(s)

        return list(results.values())
```

TIPS TO REMEMBER

Mental shortcut — “Same letters, same counts → same bucket.”

Common / related — Valid Anagram, Top K Frequent Elements, Contains Duplicate

> *"I'm turning each string into a canonical count-based fingerprint and bucketing by it in a hash map — that's a single O(n·k) pass instead of comparing every pair of strings, and the 26-slot count array keeps each fingerprint a fixed, hashable size regardless of the string's length."*

EDGE CASES

- Empty input list → return an empty list, no groups
- A single-character or single-element input → forms its own group of one
- No two strings are anagrams → n groups of one, same as the input count
- A string that's empty "" → its count array is all zeros; every other empty string groups with it
- All strings are anagrams of each other → one big group
- Input is lowercase-only per constraints — mixed case or non-letter characters would need normalizing (e.g. lowercasing) before counting